GOSAMPLES

✨ 5 different ways to loop over a time.Ticker in Go

February 27, 2023

introduction time

The `time.Ticker` is a very popular structure in Go that provides clock ticks at fixed intervals. Internally, it contains a channel that delivers subsequent ticks. Using this channel, we can create several different variants of the loop depending on the needs, for example, whether the loop should start immediately, whether we need a ticker value, or whether we want to stop the ticker properly while the program is running. All these cases will be described in this article 😊.

Simple loop over a `time.Ticker`

When you want to perform some task every fixed period of time, use the standard `for` loop over the `C` channel of `time.Ticker`. Such a loop allows you to schedule cron-like tasks or worker functions, where some code executes every specified time. Check out the example and the output. We receive a tick in the loop every second and print its time. After 5 seconds of the main thread sleeping, the ticker stops, and the app finishes.

> This type of `time.Ticker` loop is suitable for endless goroutines. However, if in a production application you allow a goroutine to terminate before the end of the program, it is better to use an example that guarantees that the goroutine will terminate correctly.

```
package main
```

Copy

```
    "time"
)

func main() {
    ticker := time.NewTicker(1 * time.Second)

    go func() {
        for t := range ticker.C {
            log.Printf("Tick at: %v\n", t.UTC())
            // do something
        }
    }()

    time.Sleep(5 * time.Second)
    ticker.Stop()
}
```

Output:

```
2022/12/06 07:05:49 Tick at: 2022-12-06 06:05:49.450700257 +0000 UTC
2022/12/06 07:05:50 Tick at: 2022-12-06 06:05:50.450801838 +0000 UTC
2022/12/06 07:05:51 Tick at: 2022-12-06 06:05:51.450907566 +0000 UTC
2022/12/06 07:05:52 Tick at: 2022-12-06 06:05:52.451003677 +0000 UTC
```

A loop without getting the tick value

In many cases, you do not need to print the tick value or use it in any other way. So you can ignore it by using the blank identifier in the loop: `for _ := range ticker.C` or skip the assignment in the loop altogether: `for range ticker.C`. Check out the example:

```
package main

import (
    "log"
    "time"
)

func main() {
    ticker := time.NewTicker(1 * time.Second)
```

```
        for range ticker.C {
            log.Println("Hey!")
        }
    }()

    time.Sleep(5 * time.Second)
    ticker.Stop()
}
```

Output:

```
2022/12/06 19:11:13 Hey!
2022/12/06 19:11:14 Hey!
2022/12/06 19:11:15 Hey!
2022/12/06 19:11:16 Hey!
```

Run `time.Ticker` loop with immediate start

Sometimes you may want the `time.Ticker` loop to return a tick immediately, without waiting for the first tick from the `C` channel. It can be useful in applications that do some work every certain amount of time, but the same work should also be done right after startup. For example, an application that fills a cache with data should do this right after the start and then at equal intervals, such as every hour. Although the `time.Ticker` does not support such an option by default, we can simulate this using the classic `for` loop form:

```
for <initialization>; <condition>; <post-condition> {
    <loopBody>
}
```

When we leave the `<condition>` empty, meaning that it will be `true`, we assure that this loop will execute at least once immediately. Subsequent executions should take place according to incoming ticks, so we need to set reading from the `Ticker.C` channel as `<post-condition>`. Run the example and look at its output. The first execution of the loop is immediate because there are no additional conditions in the `<initialization>` and `<condition>` sections, and subsequent executions happen every second according to the `<post-condition>`.

```
package main
```

```
    "time"
)

func main() {
    ticker := time.NewTicker(1 * time.Second)

    go func() {
        for ; ; <-ticker.C {
            log.Println("Hey!")
        }
    }()

    time.Sleep(5 * time.Second)
    ticker.Stop()
}
```

Output:

```
2022/12/06 19:50:37 Hey!
2022/12/06 19:50:38 Hey!
2022/12/06 19:50:39 Hey!
2022/12/06 19:50:40 Hey!
2022/12/06 19:50:41 Hey!
```

Run `time.Ticker` loop with an immediate start and get the ticker value

When you want to create a `time.Ticker` loop with the immediate first tick and use its value, you must first initialize the variable in the `for` loop and then assign successive tick values obtained from the `time.Ticker` to it. Read the example to find out how to do it:

```
package main

import (
    "log"
    "time"
)
```

```
    go func() {
        for t := time.Now(); ; t = <-ticker.C {
            log.Printf("Tick at: %v\n", t.UTC())
        }
    }()

    time.Sleep(5 * time.Second)
    ticker.Stop()
}
```

Output:

```
2022/12/07 06:08:24 Tick at: 2022-12-07 05:08:24.189026328 +0000 UTC
2022/12/07 06:08:25 Tick at: 2022-12-07 05:08:25.189313138 +0000 UTC
2022/12/07 06:08:26 Tick at: 2022-12-07 05:08:26.189548386 +0000 UTC
2022/12/07 06:08:27 Tick at: 2022-12-07 05:08:27.189798708 +0000 UTC
2022/12/07 06:08:28 Tick at: 2022-12-07 05:08:28.190043492 +0000 UTC
```

Properly finish the `time.Ticker` goroutine

When you run the `time.Ticker` loop in a goroutine, and at some point, you want to stop processing, you need to perform two steps:

- stop the `Ticker`
- ensure that the goroutine in which the `Ticker` was operating terminated correctly

To stop the `Ticker`, you need to run the `Stop()` method. However, the `Stop()` does not close the `C` channel. It only prevents new ticks from being sent. Thus, it does not automatically exit the loop and the goroutine. To close the goroutine, you need an additional signal that, when read, will cause it to close. So to achieve all this, we will use a `for` loop with a `select` statement. The `select` blocks and waits until one of its cases can run. In our situation, it should wait either for the value of the new tick from the `C` channel or for the signal to terminate the goroutine.

Look at the `main()` function in the example. After 5 seconds of goroutine processing, we stop the `Ticker` using the `Stop()` method and send a new signal to the `done` channel. As a result, the `Ticker` stops receiving new ticks in the `C` channel, and the `select` statement reads the value from the `done`, which causes the goroutine to `return`. In the first 5 seconds of the app, there are only

GOSAMPLES

```
package main

import (
    "log"
    "time"
)

func main() {
    ticker := time.NewTicker(1 * time.Second)
    done := make(chan struct{})
    go func() {
        for {
            select {
            case <-done:
                return
            case <-ticker.C:
                log.Println("Hey!")
            }
        }
    }()

    time.Sleep(5 * time.Second)
    ticker.Stop()
    done <- struct{}{}
    log.Println("Done")
}
```

Output:

```
2022/12/07 06:09:59 Hey!
2022/12/07 06:10:00 Hey!
2022/12/07 06:10:01 Hey!
2022/12/07 06:10:02 Hey!
2022/12/07 06:10:03 Hey!
2022/12/07 06:10:03 Done
```

GOSAMPLES

supporting us with a cup of coffee and we'll turn it into more great Go examples.

Have a great day!

Share:

Related

Convert date or time to string in Go

shorts introduction time

March 1, 2023

YYYY-MM-DD date format in Go

Learn how to format date without time

introduction time

March 14, 2022

Measure execution time in Go

Learn how to measure the time taken by a function

introduction time

August 10, 2021